

Chapter 2

Threads

A thread is one flow of execution: a program counter, registers and a stack. A process can contain several threads; they run the same program, share its memory and its open files, and are scheduled independently — on a multi-core machine literally at the same time. This is the abstraction you implement in SWEB for A1 (`pthread_create`, `pthread_exit`, `pthread_join`, `pthread_cancel`).

2.1 What is shared, what is private

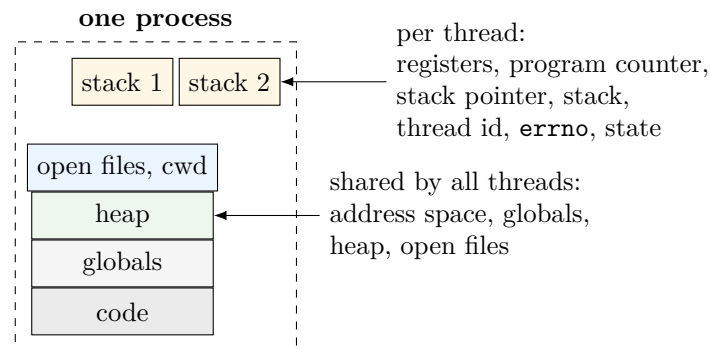


Figure 2.1: Two threads in one process.

Everything reachable through a pointer is shared — including another thread’s stack, if somebody hands out the address of a local variable. “Private” only means “not shared unless you share it”.

Thread vs. process. Creating a thread is much cheaper than creating a process: no new address space, no copied page tables, no copied descriptor table. Switching between threads of the same process does not change the page tables either. The price: no isolation. One thread’s stray write corrupts every other thread, one thread’s crash kills the whole process, and every shared variable needs synchronisation (chapters 3–5).

2.2 The pthreads interface

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                  void *(*start_routine)(void *), void *arg);
void pthread_exit(void *retval); /* never returns */
int pthread_join(pthread_t thread, void **retval);
int pthread_detach(pthread_t thread);
pthread_t pthread_self(void);
```

create starts `start_routine(arg)` in a new thread and stores its id in `*thread`. The new thread may run before or after `pthread_create` returns — never rely on either.

exit ends the calling thread with a return value. Returning from `start_routine` does the same. Returning from `main` (or calling `exit`) ends the *whole process*, all threads included.

join waits until the thread has ended and collects its return value. Afterwards the thread's resources are freed. Each thread can be joined once; joining yourself is an error (`EDEADLK`).

detach “nobody will join this thread”: its resources are freed as soon as it ends. A detached thread cannot be joined.

All functions return 0 or an error number (they do not set `errno`).

2.3 The lifecycle

A thread is created *ready*, becomes *running* when the scheduler picks it, may *block* (waiting for a lock, for I/O, in `join`) and becomes ready again when woken, and finally *terminates*. A terminated thread that is joinable but not yet joined is a *zombie*: its stack can go, but its return value and id must stay until somebody joins it — exactly like a zombie process (chapter 1). Neither joined nor detached means leaked: the zombie stays forever. Chapter 10 builds this state machine.

2.4 Passing arguments and results

Arguments. `arg` is a single `void *`. For more than one value, pass a pointer to a struct — and make sure the struct lives long enough:

```
struct job { int first, count; long result; };
struct job jobs[N];                               /* one per thread, in the caller's frame */
for (int i = 0; i < N; i++) {
    jobs[i] = (struct job){i * chunk, chunk, 0};
    pthread_create(&tid[i], NULL, worker, &jobs[i]); /* not &i ! */
}
for (int i = 0; i < N; i++)
    pthread_join(tid[i], NULL);                     /* jobs[] must outlive the threads */
```

Two classic bugs (module 02, part B): passing `&i` (every thread gets the same, changing address), and passing a pointer to a local of a function that returns before the threads read it.

Results. Either the thread writes into its own struct (as above — no sharing, no lock needed: each thread owns one element), or it returns a pointer. Never return the address of a local; return heap memory, a pointer into the argument, or a small integer cast to `void *`.

Partition, then combine. The simplest correct parallel program splits the data into disjoint parts, lets each thread work only on its part, and combines the partial results *after* `pthread_join`. No shared writes, no locks — and `pthread_join` guarantees that the joiner sees everything the thread wrote (a happens-before edge, chapter 3).

2.5 Thread-local data

Some state must exist once per thread although it looks global. The classic example is `errno`: if thread A's failing call and thread B's failing call wrote the same variable, A could read B's error. C11 has `_Thread_local` (GCC: `__thread`); glibc keeps `errno` in thread-local storage. module 07's mini libc makes its `ml_errno` thread-local for exactly this reason.

2.6 Who implements the threads?

Kernel-level threads (1:1). Every user thread is a thread the kernel knows and schedules.

Blocking in a system call blocks only that thread; threads run in parallel on several cores. Every switch goes through the kernel. Linux (NPTL), Windows — and SWEB after A1.

User-level threads (M:1). A library switches between threads inside one kernel thread (module 10). Switching is a function call, but a blocking system call blocks all threads, and only one core is used.

Hybrid (M:N). M user threads on N kernel threads; complex and mostly abandoned (Go's runtime is a modern variant).

In SWEB

The stubs are in `userspace/libc/src/pthread.c`: `pthread_create` just `return -1`; P1 turns it into a system call that creates a new kernel `Thread` in the calling process (1:1): it gets its own kernel stack (inside the `Thread` object, `kernel_stack_`), its own user stack in the shared address space, and its own saved registers — and starts in user mode at a small libc wrapper that calls `start_routine(arg)` and then `pthread_exit` (chapter 10).

Key points

- Threads share the address space and open files; each has its own registers and stack.
- Returning from the start routine = `pthread_exit`; returning from `main` ends every thread.
- Join or detach every thread; an unjoined finished thread is a zombie.
- Pass each thread its own argument object that outlives it; never `&i`, never a dying local.
- Partition-then-combine needs no locks; `join` makes the results visible.